

Relatório 13 - Prática: Projeto Final - Part 2

José Carlos Seben de Souza Leite

Descrição da atividade

Dando continuidade na elaboração do projeto final agora construindo um backend para posteriormente o anexar com o frontend que já temos construído o planejamento aqui foi utilizando as tecnologias utilizadas durante o fastcamp e requisitadas na descrição do card construir uma API que atendesse todos os requisitos e já tivesse estrutura preparada para atender o que o frontend construído na parte 1.

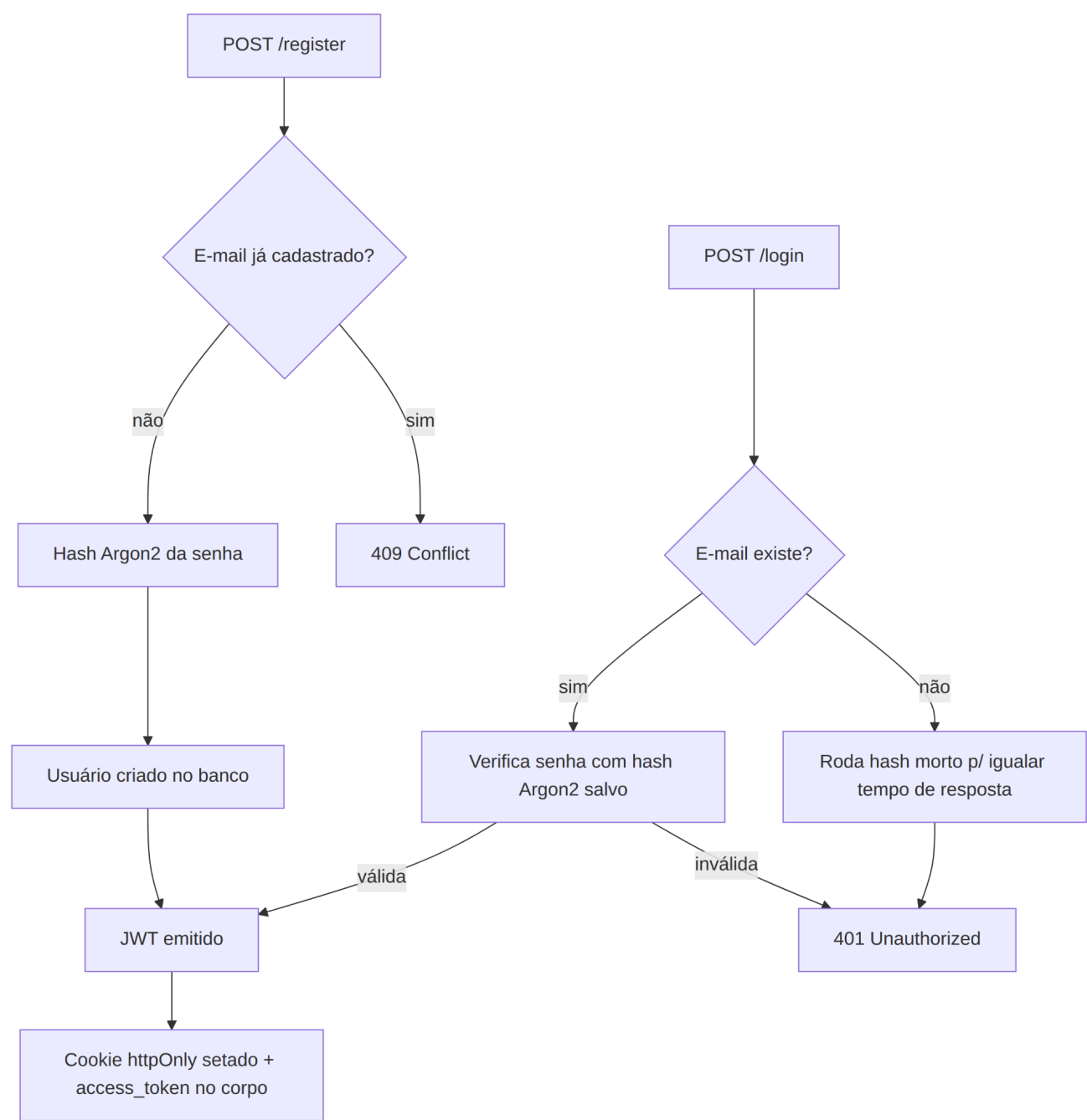
Para a stack, seguindo o que foi sugerido no card, usamos FastAPI como framework, PostgreSQL como banco, JWT para a autenticação, Pydantic para validação e FastAPI-Mail para o envio dos e-mails. A baixo vou descrever melhor como cada uma dessas ferramentas foi usada e como elas se integram, mas de forma resumida a organização do backend foi feita separando bem as responsabilidades em camadas para não virar um arquivo gigante fazendo tudo.

- **FastAPI:** framework principal da API, escolhido por já ter sido usado nos cards anteriores de backend e por ser requisito do card. Além de montar as rotas, ele já entrega de graça a documentação interativa no /docs (Swagger), o que ajudou muito na hora de testar.
- **PostgreSQL + SQLAlchemy:** o banco de dados relacional pedido no card, acessado pelo SQLAlchemy 2.0 (usando o mapeamento por dataclass) e com o Alembic cuidando das migrações, ou seja, das mudanças de estrutura do banco sendo versionadas em vez de rodar SQL na mão.
- **JWT:** usado para a sessão do usuário. Cada login gera um token assinado que carrega quem é o usuário, e o servidor valida esse token nas rotas protegidas para saber quem está chamando, sem precisar guardar sessão em memória.
- **Pydantic:** usado para a validação de entrada e saída de dados, definindo regras como e-mail em formato válido, senha entre 6 e 128 caracteres, confirmação de senha igual à senha e código de recuperação com exatamente 6 dígitos, tudo isso já validado antes da rota rodar. Foi o mesmo papel que o Zod fez lá no frontend da parte um.
- **FastAPI-Mail:** usado no fluxo de recuperação de senha para enviar o código de 6 dígitos por e-mail. Em desenvolvimento o envio vem desligado por padrão (MAIL_SUPPRESS_SEND), então dá pra testar o fluxo inteiro sem precisar configurar um servidor SMTP de verdade.

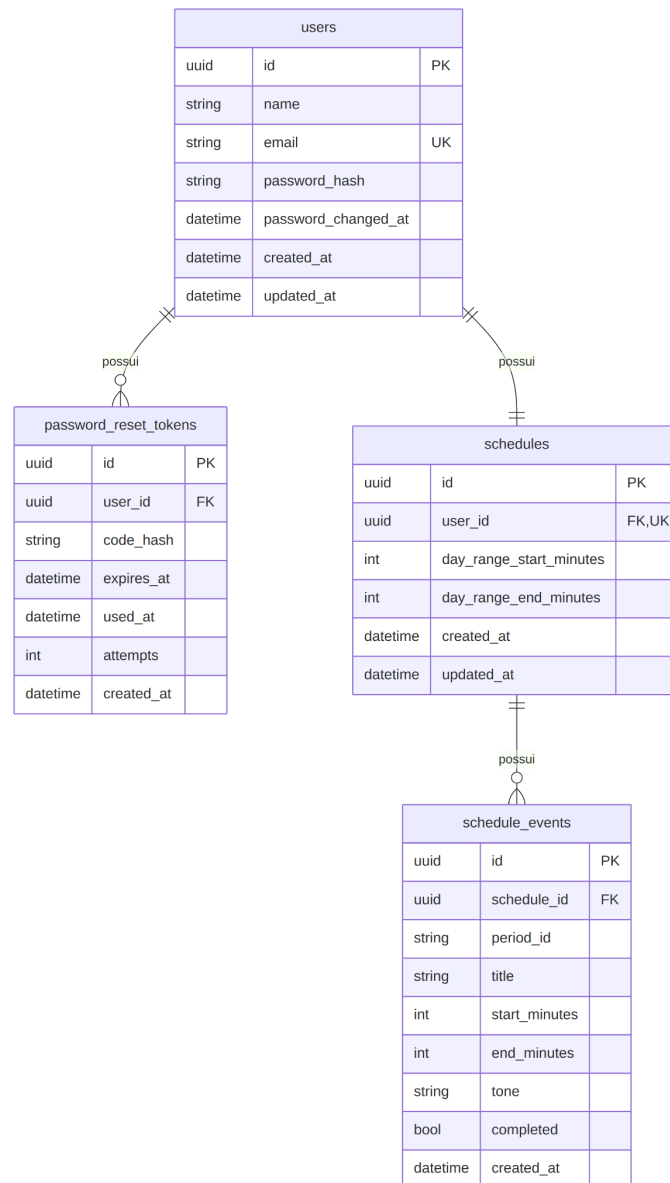
Além dessas, para o hash de senha usei o pwdlib com Argon2, e para os testes automatizados o pytest com pytest-cov. Também deixei tudo dockerizado com Docker Compose para subir a API e o Postgres com um comando só, o que facilita bastante a vida sem precisar instalar Postgres na máquina.

A arquitetura seguiu uma separação simples por camadas, que foi a forma que achei mais organizada pensando em cada parte ter uma responsabilidade só. As rotas ficam finas dentro da pasta routers/, as regras de autenticação e de hash de senha ficam isoladas no security.py, os schemas de entrada e saída no schemas.py e o acesso ao banco centralizado no database.py junto do models.py. A regra que tentei seguir foi que nenhuma rota monta SQL na mão nem mexe em hash direto, cada router só conhece os schemas e as funções do security.py, assim se eu precisar mudar como o hash funciona, por exemplo, mudo num lugar só.

O fluxo de login ficou estabelecido na seguinte ordem:



O banco é PostgreSQL e tem quatro tabelas, criadas por migrações do Alembic em vez de rodar SQL na mão. São elas: users, password_reset_tokens, schedules e schedule_events.



As relações ficaram assim:

- **users** → **password_reset_tokens (1:N)**: um usuário pode ter vários códigos de recuperação ao longo do tempo. A tabela guarda só o hash do código (nunca o código puro), marca quando ele foi usado e conta as tentativas.
- **users** → **schedules (1:1)**: cada usuário tem uma única agenda, criada no primeiro acesso. Ela guarda basicamente o intervalo do dia que fica visível na timeline, em minutos desde a meia-noite.
- **schedules** → **schedule_events (1:N)**: cada card de evento pertence a uma agenda e carrega o próprio horário, o período do dia em que cai (manhã, almoço, tarde ou noite, calculado no backend) e se já foi concluído.

O e-mail do usuário é único e normalizado pra minúsculo antes de salvar, pra não dar para criar duas contas com o mesmo e-mail só mudando a caixa das letras. E a coluna que guarda a data da última troca de senha é o que faz as sessões antigas caírem: o JWT carrega essa data, e qualquer token emitido antes de uma troca deixa de valer.

A maneira mais fácil de rodar é pelo Docker, que sobe o banco e a API juntos. O passo a passo pra quem baixar o repositório é:

1. Colocar o arquivo .env na raiz da pasta do Card 13. Esse .env foi passado direto pro orientador, já com as variáveis preenchidas, então é só usar o que foi enviado (não precisa configurar nada na mão).
2. Subir tudo com docker compose up -d --build. O backend espera o Postgres ficar saudável e roda as migrações do Alembic automaticamente antes de subir o servidor.
3. Acessar a documentação em <http://localhost:8001/docs> e testar os endpoints por ali pelo Swagger.

Conclusões

Agora esperar a correção para iniciarmos a junção entre front e back e se necessários as adaptações para juntar elas e concluir o fastcamp.